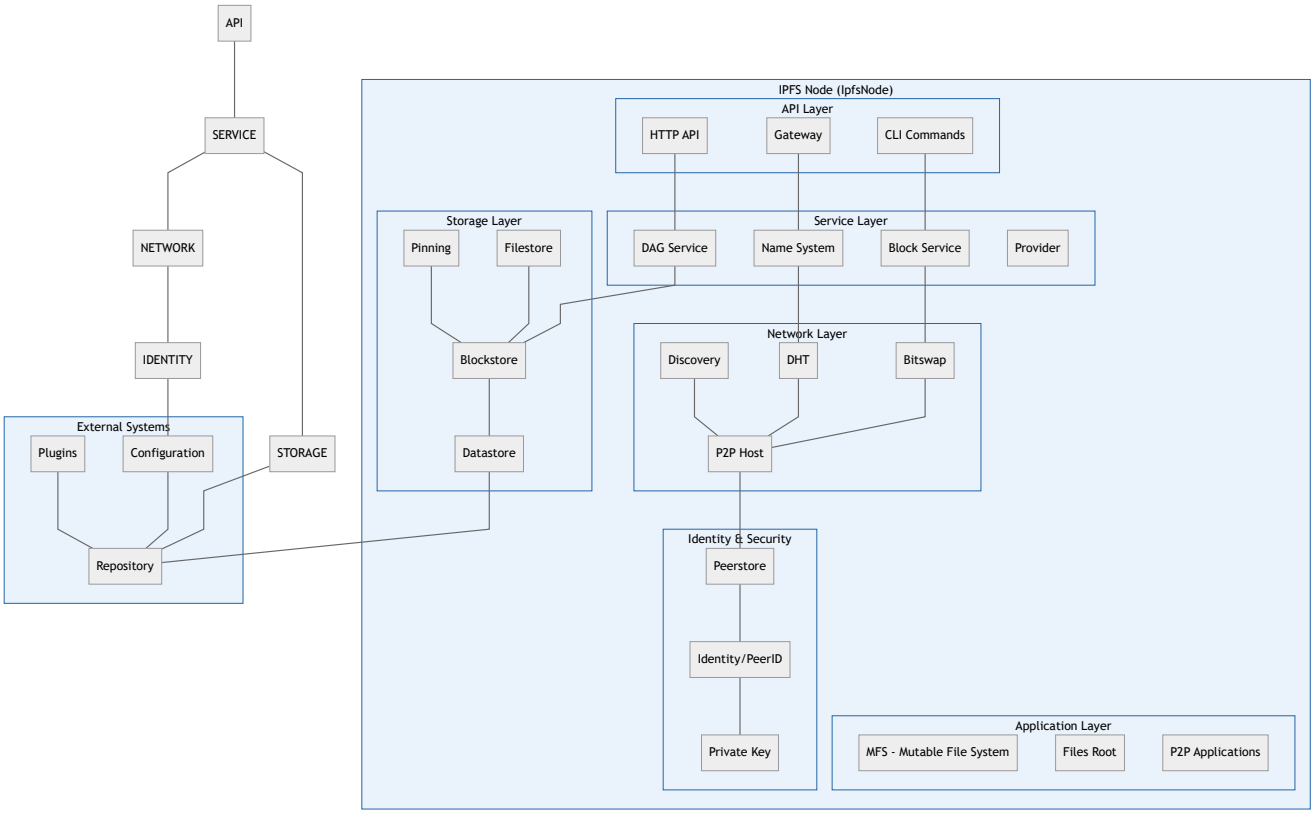
IPFS 전체 아키텍처 개요

소개

IPFS(InterPlanetary File System)는 분산형 파일 시스템으로, 콘텐츠 주소 지정을 통해 데이터를 저장하고 공유하는 P2P 네트워크입니다. Kubo는 IPFS의 Go 언어 구현체로, 모듈화된 아키텍처를 통해 다양한 컴포넌트들이 협력하여 분산 파일 시스템을 구현합니다.

시스템 아키텍처 다이어그램


API
SERVICE
NETWORK
IDENTITY
External Systems
Plugins
Configuration
Repository
STORAGE
IPFS Node (IpfsNode)
API Layer
HTTP API
Gateway
CLI Commands
Storage Layer
Pinning
Filestore
Blockstore
Datastore
Service Layer
DAG Service
Name System
Block Service
Provider
Network Layer
Discovery
DHT
Bitswap
P2P Host
Identity & Security
Peerstore
Identity/PeerID
Private Key
Application Layer
MFS - Mutable File System
Files Root
P2P Applications

핵심 컴포넌트 개요

1. IpfsNode - 중앙 노드 구조체

`IpfsNode` 는 IPFS 인스턴스의 핵심으로, 모든 주요 컴포넌트들을 포함하고 관리합니다:

```
type IpfsNode struct {
// 신원 관리
Identity peer.ID
PrivateKey ic.PrivKey
Peerstore pstore.Peerstore
// 저장소 계층
Blockstore bstore.GCBlockstore
Filestore *filestore.Filestore
Pinning pin.Pinner
// 네트워크 계층
PeerHost p2phost.Host
DHT *ddht.DHT
Bitswap *bitswap.Bitswap
// 서비스 계층
DAG ipld.DAGService
Blocks bserv.BlockService
Namesys namesys.NameSystem
Provider provider.System
```

```
// 애플리케이션 계층
FilesRoot *mfs.Root
P2P *p2p.P2P
}
```

2. 주요 컴포넌트별 역할

신원 및 보안 (Identity & Security)

- **Identity (peer.ID)**: 노드의 고유 식별자
- **PrivateKey**: 노드의 개인키, 서명 및 암호화에 사용
- **Peerstore**: 다른 피어들의 정보를 저장하는 저장소

저장소 계층 (Storage Layer)

- **Blockstore**: 블록 데이터의 물리적 저장소
- **Datastore**: 메타데이터와 인덱스 정보 저장
- **Filestore**: 파일 시스템 기반 블록 저장소 (선택적)
- **Pinning**: 중요한 콘텐츠를 로컬에 고정하는 시스템

네트워크 계층 (Network Layer)

- **P2P Host**: libp2p 기반 네트워크 호스트
- **DHT (Distributed Hash Table)**: 분산 해시 테이블, 피어 및 콘텐츠 발견
- **Bitswap**: 블록 교환 프로토콜
- **Discovery**: 피어 발견 메커니즘 (mDNS 등)

서비스 계층 (Service Layer)

- **DAG Service**: IPLD DAG 구조 관리
- **Block Service**: 블록 검색 및 저장 서비스
- **Name System**: IPNS (InterPlanetary Name System) 구현
- **Provider**: 콘텐츠 제공자 시스템

API 계층 (API Layer)

- **HTTP API**: RESTful HTTP 인터페이스
- **CLI Commands**: 명령줄 인터페이스
- **Gateway**: HTTP 게이트웨이 서비스

애플리케이션 계층 (Application Layer)

- **MFS (Mutable File System)**: 변경 가능한 파일 시스템 인터페이스
- **Files Root**: 파일 시스템 루트 관리
- **P2P Applications**: P2P 애플리케이션 터널링

컴포넌트 간 상호작용

데이터 흐름

1. **데이터 추가 (Add) 흐름**:

```
CLI/HTTP API → Block Service → DAG Service → Blockstore → Datastore
↓
Provider System → DHT → Network (Bitswap)
```

2. **데이터 검색 (Get) 흐름**:

```
CLI/HTTP API → DAG Service → Block Service → Blockstore (로컬)
↓ (없으면)
Bitswap → DHT → Network → Remote Peers → Blockstore (저장)
```

3. **피어 발견 흐름**:

```
Discovery (mDNS) → Peerstore → P2P Host → DHT → Bootstrap Peers
```

의존성 관계

- **상위 계층**은 **하위 계층**에 의존
- **API 계층 → 서비스 계층 → 네트워크/저장소 계층 → 신원 계층**
- **횡적 의존성**: 같은 계층 내 컴포넌트들 간의 상호작용

초기화 및 생명주기

노드 초기화 과정

1. **설정 로드**: Repository에서 설정 정보 읽기
2. **신원 설정**: PeerID와 개인키 로드/생성
3. **저장소 초기화**: Blockstore, Datastore 설정
4. **네트워크 설정**: libp2p Host 및 프로토콜 초기화
5. **서비스 시작**: 각종 서비스들 순차적 시작
6. **부트스트랩**: 네트워크 연결 및 피어 발견

의존성 주입 (Dependency Injection)

IPFS는 Uber의 FX 프레임워크를 사용하여 의존성 주입을 구현합니다:

```
// 주요 컴포넌트 그룹들
var Core = fx.Options(
fx.Provide(Dag),
fx.Provide(FetcherConfig),
fx.Provide(PathResolverConfig),
)

func Online(bcfg *BuildCfg, cfg *config.Config) fx.Option {
return fx.Options(
fx.Provide(Bitswap),
fx.Provide(OnlineExchange),
fx.Provide(DNSResolver),
fx.Provide(Namesys),
LibP2P(bcfg, cfg),
// ...
)
}
```

컴포넌트 그룹화

1. **Identity**: 신원 관련 컴포넌트
2. **Storage**: 저장소 관련 컴포넌트

3. **LibP2P**: 네트워크 관련 컴포넌트
4. **Core**: 핵심 서비스 컴포넌트
5. **Online/Offline**: 온라인/오프라인 모드별 컴포넌트

설정 및 확장성

모듈화 설계

- **플러그인 시스템**: 다양한 백엔드 구현체 지원
- **인터페이스 기반**: 구현체 교체 가능
- **설정 기반**: 런타임 동작 제어

확장 포인트

- **Blockstore 백엔드**: Badger, LevelDB, Flatfs 등
- **Routing 구현체**: DHT, Delegated Routing 등
- **Exchange 프로토콜**: Bitswap, HTTP Retrieval 등
- **Transport 프로토콜**: TCP, WebSocket, QUIC 등

성능 및 최적화

캐싱 전략

- **Block Cache**: 자주 사용되는 블록 캐싱
- **DHT Cache**: 라우팅 정보 캐싱
- **Name Resolution Cache**: IPNS 해석 결과 캐싱

리소스 관리

- **Connection Manager**: 연결 수 제한 및 관리
- **Resource Manager**: 메모리 및 네트워크 리소스 제한
- **Garbage Collection**: 사용하지 않는 블록 정리

이 아키텍처를 통해 IPFS는 확장 가능하고 모듈화된 분산 파일 시스템을 구현하며, 각 컴포넌트의 독립성과 상호 운용성을 보장합니다.